

Model ensembles and mixtures of experts

MIPT

2025

Model ensembling

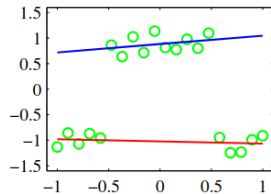
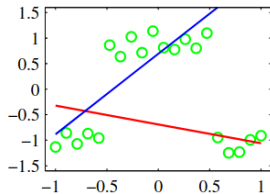
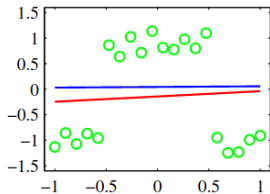
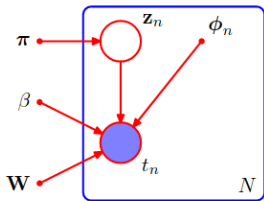
Definition (Wiki)

In statistics and machine learning, ensemble methods use multiple learning algorithms to obtain better predictive performance than could be obtained from any of the constituent learning algorithms alone. A machine learning ensemble consists of only a concrete finite set of alternative models, but typically allows for much more flexible structure to exist among those alternatives.

Naive ensembling

Mixture model

$$f = \sum \gamma_i f_i(x)$$



Model selection: coherent Bayesian inference

First level: find optimal parameters:

$$w = \arg \max \frac{p(\mathcal{D}|w)p(w|h)}{p(\mathcal{D}|h)},$$

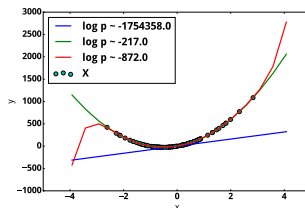
Second level: find optimal model:

Evidence:

$$p(\mathcal{D}|h) = \int_w p(\mathcal{D}|w)p(w|h)dw.$$



Model selection scheme



Polynomial regression example

Mixutre vs Bayesian model averaging

Mixture:

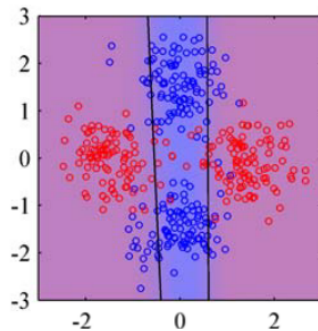
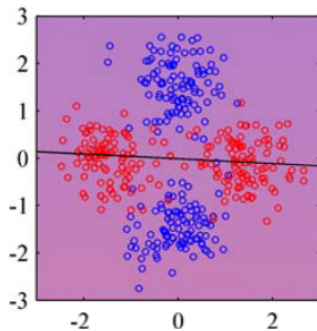
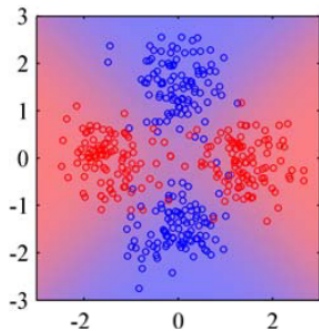
$$f = \sum \gamma_i f_i(X) = \sum \prod_x p(x, \gamma_i)$$

Averaging:

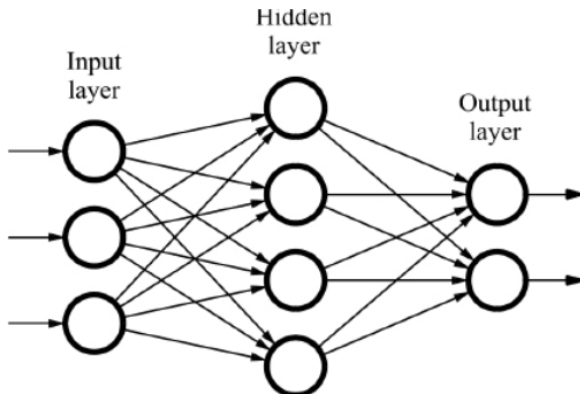
$$f = \sum p(f_i) f_i(X).$$

Mixture of experts

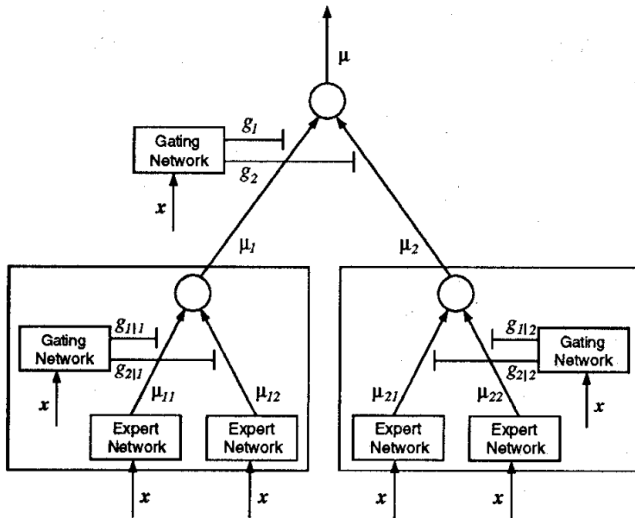
$$f = \sum \gamma_i(x) f_i(x)$$



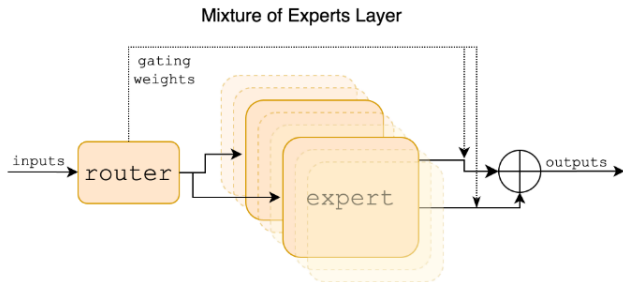
MLP: mixture?



Mixutre of experts with hierarchy



MOE in Mixtral



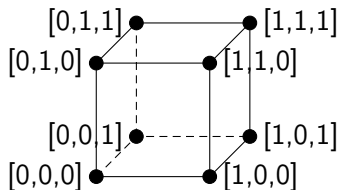
- Per-token experts
- Gate: Softmax over top-k weights
- Top-k enforces sparsification → less resources are used for inference

Multimodels

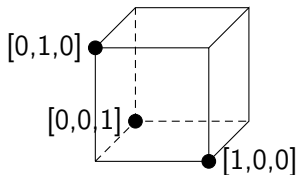
$$f = \sum \gamma_i(x) f_i(x),$$
$$\sum \gamma = 1, \quad \gamma_i \in 0, 1.$$

Structure restrictions

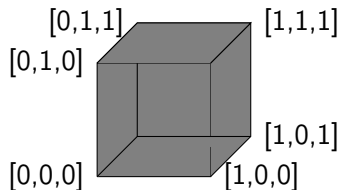
An example of restrictions for structure parameter γ , $|\gamma| = 3$.



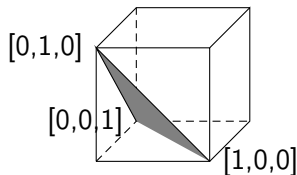
Cube vertices



Simplex vertices



Cube interior

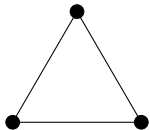


Simplex interior

Prior distribution for the model structure

Every point in a simplex defines a model.

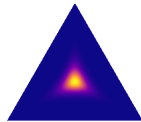
Gumbel-Softmax distribution: $\boldsymbol{\Gamma} \sim \text{GS}(\mathbf{s}, \lambda_{\text{temp}})$



$$\lambda_{\text{temp}} \rightarrow 0$$

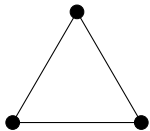


$$\lambda_{\text{temp}} = 0.995$$



$$\lambda_{\text{temp}} = 5.0$$

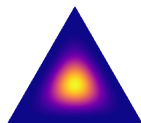
Dirichlet distribution: $\boldsymbol{\Gamma} \sim \text{Dir}(\mathbf{s}, \lambda_{\text{temp}})$



$$\lambda_{\text{temp}} \rightarrow 0$$

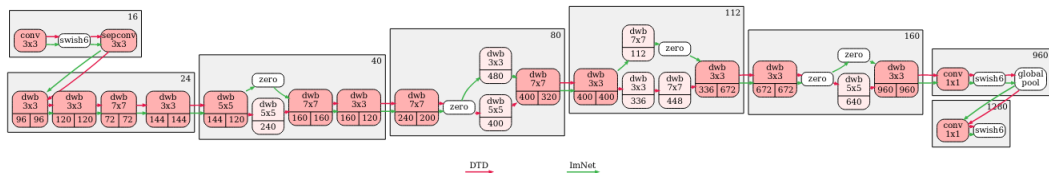


$$\lambda_{\text{temp}} = 0.995$$



$$\lambda_{\text{temp}} = 5.0$$

Multi-domain tasks



Uncertainty estimation, Depeweg et al., 2017¹

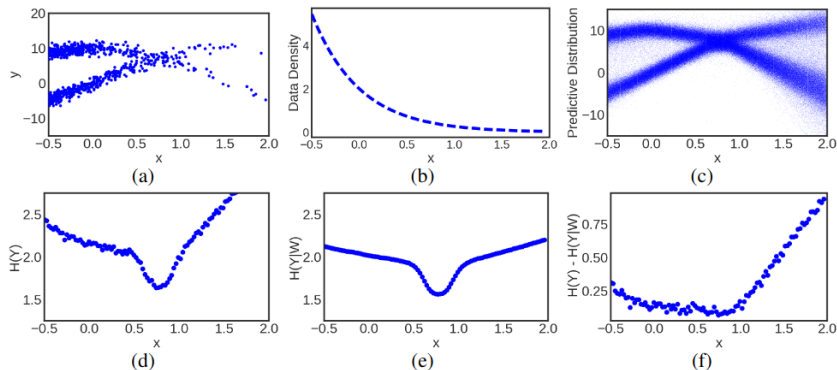


Figure 2. Uncertainty decomposition on bimodal data. (a): Raw data. (b): Density of x in data. (c): Predictive distribution: $p(y_\star | x_\star)$. (d): Estimate of $H(y_\star | x_\star)$. (e): Estimate of $\mathbf{E}_{q(\mathcal{W})} [H(y_\star | x_\star, \mathcal{W})]$. (f): Estimate of entropy reduction $H(y_\star | x_\star) - \mathbf{E}_{q(\mathcal{W})} [H(y_\star | x_\star, \mathcal{W})]$.

¹See the talk of Maxim Panov: <https://intsystems.github.io/materials/seminars/>.

Uncertainty estimation using ensembles

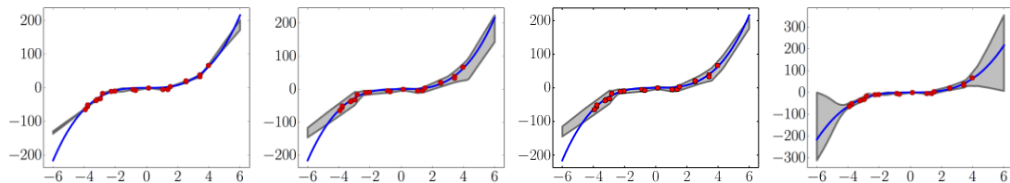
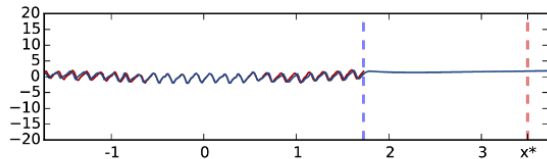


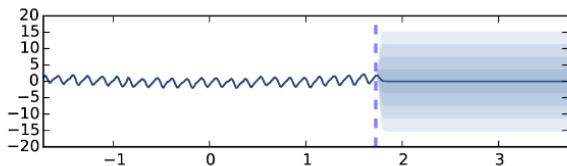
Figure 1: Results on a toy regression task: x -axis denotes x . On the y -axis, the blue line is the *ground truth* curve, the red dots are observed noisy training data points and the gray lines correspond to the predicted mean along with three standard deviations. Left most plot corresponds to empirical variance of 5 networks trained using MSE, second plot shows the effect of training using NLL using a single net, third plot shows the additional effect of adversarial training, and final plot shows the effect of using an ensemble of 5 networks respectively.

Uncertainty estimation: MC Dropout

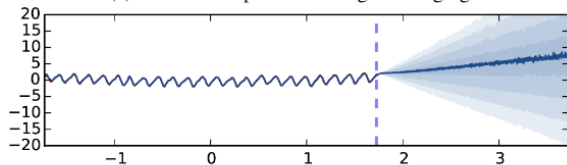
- Variational dropout: dropout can be considered as an ensemble of models²
- MC Dropout: variance of prediction model is an approximation of uncertainty
 - ▶ Which type of uncertainty?



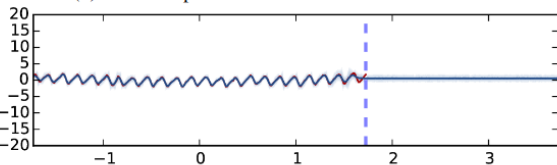
(a) Standard dropout with weight averaging



(b) Gaussian process with SE covariance function



(c) MC dropout with ReLU non-linearities



(d) MC dropout with TanH non-linearities

²See the talk of Eduard Vladimirov

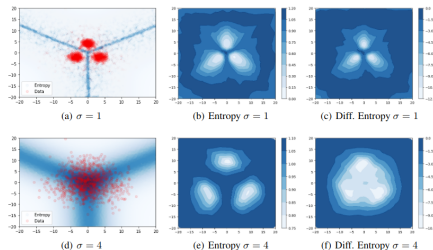
Prior network³

Basic idea: instead of training a probabilistic model that approximates $\int_w p(y|X, w)p(w)dw$ we extend it in the following way:

$$\int_{w, \mu} p(y|\mu)p(\mu|w, X)p(w)dw d\mu.$$

The model is trained with prior:

- $\mu \sim \text{Dir}(\alpha, t)$, $t \approx 0$ for ordinary dataset objects;
- $\mu \sim \mathcal{U}$ for OOD objects.



³See the talk of Gregory Ksenofontov

Neural ensemble search

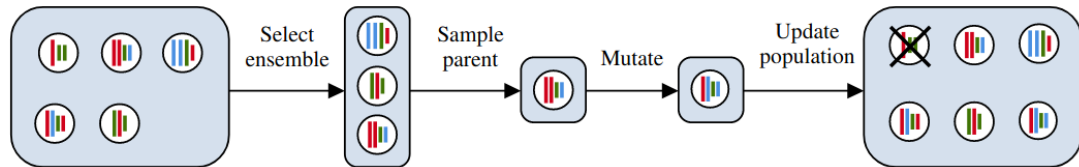


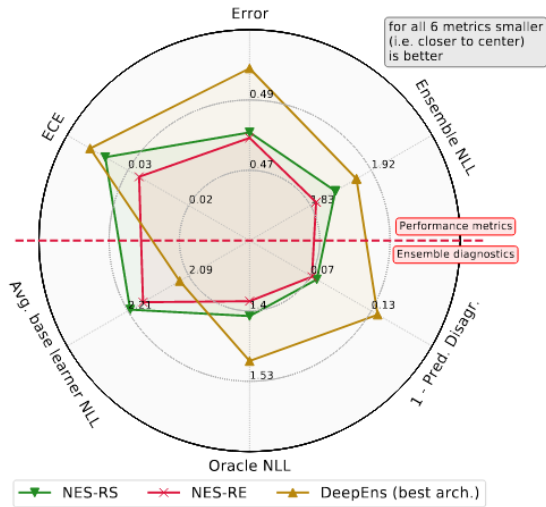
Figure 3: Illustration of one iteration of NES-RE. Network architectures are represented as colored bars of different lengths illustrating different layers and widths. Starting with the current population, ensemble selection is applied to select parent candidates, among which one is sampled as the parent. A mutated copy of the parent is added to the population, and the oldest member is removed.

Neural ensemble search

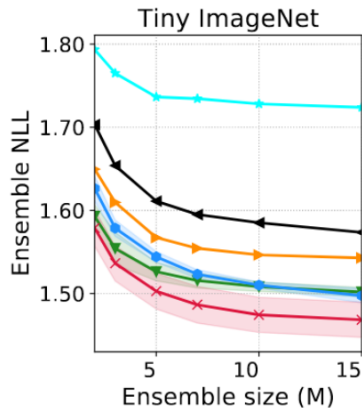
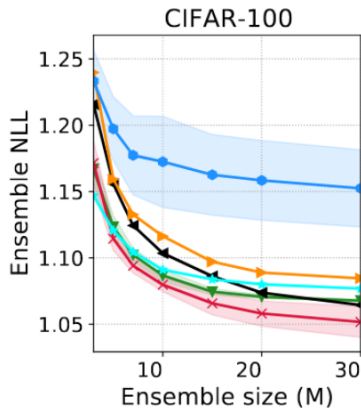
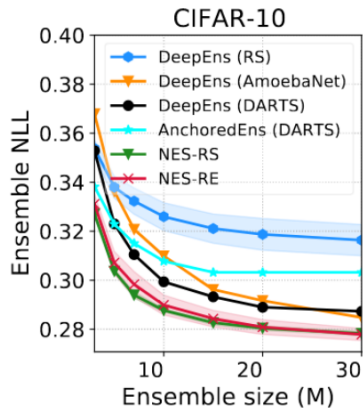
Algorithm 1: NES with Regularized Evolution

- Data:** Search space $\mathcal{A}$; ensemble size M ; comp. budget K ; $\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}$; population size P ; number of parent candidates m .
- 1 Sample P architectures $\alpha_1, \dots, \alpha_P$ independently and uniformly from $\mathcal{A}$.
 - 2 Train each architecture α_i using $\mathcal{D}_{\text{train}}$, and initialize $\mathbf{p} = \mathcal{P} = \{f_{\theta_1, \alpha_1}, \dots, f_{\theta_P, \alpha_P}\}$.
 - 3 **while** $|\mathcal{P}| < K$ **do**
 - 4 Select m parent candidates $\{f_{\tilde{\theta}_1, \tilde{\alpha}_1}, \dots, f_{\tilde{\theta}_m, \tilde{\alpha}_m}\} = \text{ForwardSelect}(\mathbf{p}, \mathcal{D}_{\text{val}}, m)$.
 - 5 Sample uniformly a parent architecture α from $\{\tilde{\alpha}_1, \dots, \tilde{\alpha}_m\}$. // α stays in $\mathbf{p}$.
 - 6 Apply mutation to α , yielding child architecture β .
 - 7 Train β using $\mathcal{D}_{\text{train}}$ and add the trained network $f_{\theta, \beta}$ to $\mathbf{p}$ and $\mathcal{P}$.
 - 8 Remove the oldest member in $\mathbf{p}$. // as done in RE [49].
 - 9 Select base learners $\{f_{\theta_1^*, \alpha_1^*}, \dots, f_{\theta_M^*, \alpha_M^*}\} = \text{ForwardSelect}(\mathcal{P}, \mathcal{D}_{\text{val}}, M)$ by forward step-wise selection without replacement.
 - 10 **return** ensemble $\text{Ensemble}(f_{\theta_1^*, \alpha_1^*}, \dots, f_{\theta_M^*, \alpha_M^*})$
-

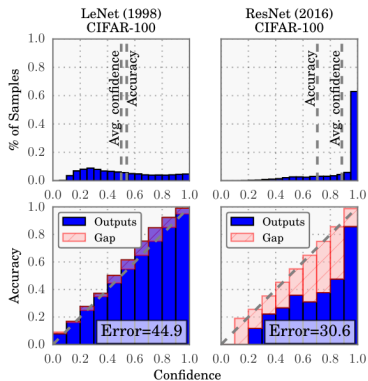
Neural ensemble search



Neural ensemble search

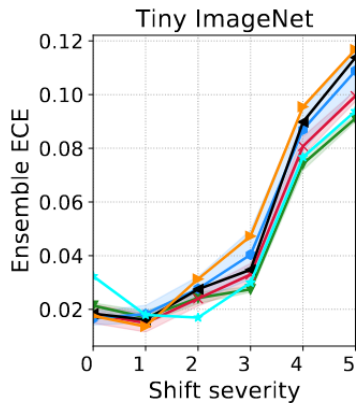
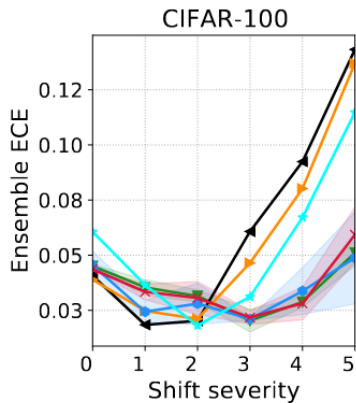
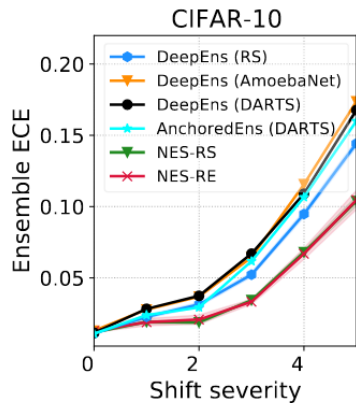


Neural ensemble search



$$\mathbb{E}_{\hat{P}} \left[\left| \mathbb{P} \left(\hat{Y} = Y \mid \hat{P} = p \right) - p \right| \right]$$
$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{n} \left| \text{acc}(B_m) - \text{conf}(B_m) \right|,$$

Neural ensemble search



Bias-variance relation in ensembles

(Krogh, Vedelsby, 1995):

Ensemble error = Average error – Ensemble response variance.

- High bias: all responses are correlated $\rightarrow$ ensemble error is the average of errors of the ensemble elements.
- High variance: Ambiguity (variance) is high $\rightarrow$ ensemble error become smaller.
- Works only with L2 loss for regression.

Bias-variance relation in ensembles, unified view

(Wood et al., 2023):

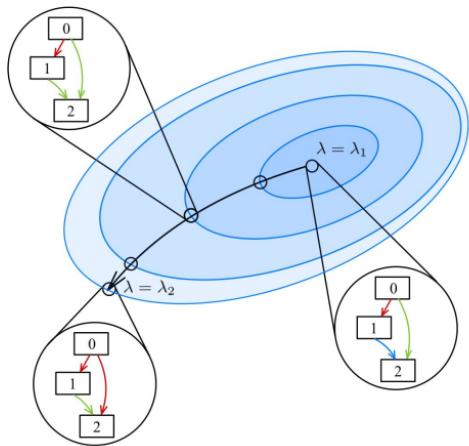
$$\text{Ensemble error} = E_x \left(E_{y|x} l(y^*|y) + \frac{1}{K} \sum_k l(C_y, y^k) - \frac{1}{K} l(\bar{y}, y^k) \right).$$

Here C_y is the geometric mean of the predictions, $\bar{y}$ is average prediction.

Second term is variance, last term is **diversity**.

- Unregularized case: bias and variance increases with model capacity increase $\rightarrow$: error potentially can increase too.
- **BUT**: diversity can compensate this issue.

NES with diversity restriction



- Structures are generated by the hypernetwork
- Hypernetwork works w.r.t. diversity parameter
- Multiple ways how to empirically measure the diversity

References

- Bishop C. M., Nasrabadi N. M. Pattern recognition and machine learning. – New York : springer, 2006. – Т. 4. – №. 4. – С. 738.
- Адуенко А. А. 2017. Выбор мультимоделей в задачах классификации. Диссертация.
- Jordan M. I., Jacobs R. A. Hierarchical mixtures of experts and the EM algorithm //Neural computation. – 1994. – Т. 6. – №. 2. – С. 181-214.
- Бахтеев О. Ю. 2020. Байесовский выбор субоптимальной структуры модели глубокого обучения. Диссертация.
- Depeweg, Stefan, et al. "Decomposition of uncertainty in Bayesian deep learning for efficient and risk-sensitive learning." International conference on machine learning. PMLR, 2018.
- Wang Q. et al. Multi-path neural networks for on-device multi-domain visual classification //Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision. – 2021. – С. 3019-3028.
- Liu H., Simonyan K., Yang Y. Darts: Differentiable architecture search //arXiv preprint arXiv:1806.09055. – 2018.
- Lakshminarayanan B., Pritzel A., Blundell C. Simple and scalable predictive uncertainty estimation using deep ensembles //Advances in neural information processing systems. – 2017. – Т. 30.
- Zaidi S. et al. Neural ensemble search for uncertainty estimation and dataset shift //Advances in Neural Information Processing Systems. – 2021. – Т. 34. – С. 7898-7911.
- Guo C. et al. On calibration of modern neural networks //International conference on machine learning. – PMLR, 2017. – С. 1321-1330
- Kingma, Durk P., Tim Salimans, and Max Welling. "Variational dropout and the local reparameterization trick." Advances in neural information processing systems 28 (2015).
- Gal, Yarin, and Zoubin Ghahramani. "Dropout as a bayesian approximation: Representing model uncertainty in deep learning." international conference on machine learning. PMLR, 2016.
- Malinin, Andrey, and Mark Gales. "Predictive uncertainty estimation via prior networks." Advances in neural information processing systems 31 (2018).
- Krogh, A., and Vedelsby, J. (1994). Neural network ensembles, cross validation, and active learning. Advances in neural information processing systems, 7.
- Wood, D., Mu, T., Webb, A. M., Reeve, H. W., Lujan, M., and Brown, G. (2023). A unified theory of diversity in ensemble learning. Journal of machine learning research, 24(359), 1-49.

Organizational issues

- General requirements for library and algorithms:
 - ▶ Codestyle: at least PEP-8
 - ▶ The code must be commented
 - ▶ There must be an installation file (setup.py or alternative)
 - ▶ There must be requirements file with defined versions
 - ▶ All these points (and general work of library) will be checked offline
- Demo: must be available at the meeting (perfect case: ipynb or collab)
- Tests:
 - ▶ Coverage 75%;
 - ▶ Tests during build: not required
 - ▶ Will be checked offline
- Documentation: must be ready and deployed
- Blog-post: pre-final version must be ready
- Documentation and blog-post will be checked offline. For blog-post be prepared for the external review.